

From Einstein-Boltzmann Equations to Galaxy Power Spectra: Physicist-Supervised, Oracle-Guided AI Reimplementation in JAX

Anonymous Authors¹

Abstract

Joint analyses of cosmic microwave background (CMB) and galaxy survey data increasingly rely on gradient-based sampling, yet no existing code computes both CMB angular power spectra and galaxy power spectrum multipoles from first principles with exact automatic differentiation on GPU. We present CLAX and CLAX-PT—JAX reimplementations of the CLASS Boltzmann solver and its one-loop perturbation theory extension CLASS-PT—developed by physicist-supervised, oracle-guided AI agents that iterate against CLASS and CLASS-PT reference outputs and source codes. Agents autonomously resolved $\sim 90\%$ of documented bugs; the remaining $\sim 10\%$ required human physics judgment, falling into two failure modes: *architectural incoherence* (code structure incompatible with the target physics) and *oracle-passing fudge factors* (a numerical patch satisfying all tests but encoding no physical model). CLAX achieves $< 0.2\%$ lensed CMB accuracy at $\ell \leq 2000$; CLAX-PT achieves $< 0.7\%$ accuracy for galaxy power spectrum monopoles and quadrupoles at $k < 0.3 \text{ h/Mpc}$. Automatic-differentiation gradients agree with finite differences to 0.15% on average. We propose three principles for structuring physicist-agent collaboration in scientific software development and compare against concurrent differentiable codes DISCO-EB, ABCMB, and SYMBOLTZ.JL.

1. Introduction

The standard model of cosmology is tested by comparing theoretical predictions against observations of the cosmic microwave background (CMB) angular power spectra from

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the AI4Physics Workshop at the International Conference on Machine Learning (AI4Physics@ICML 2026). Do not distribute.

CMB experiments (e.g., Aghanim et al., 2020) and of the galaxy power spectrum multipoles from galaxy redshift surveys (e.g., Adame et al., 2025b;a). This comparison increasingly benefits from gradient-based inference: forecasts require likelihood derivatives, Hamiltonian Monte Carlo analyses require posterior derivatives. Gradient-based inference libraries such as NUMPYRO (Phan et al., 2019) and BLACKJAX (Cabezas et al., 2024) but the missing ingredient is a differentiable cosmological forward model that can be passed to them.

Standard Boltzmann solvers for CMB angular power spectra, e.g., CLASS (Lesgourgues, 2011), and one-loop perturbation theory codes for galaxy power spectrum multipoles, e.g., CLASS-PT, are written in C (or Fortran), are not differentiable, and are not GPU-native. Several JAX implementations exist but cover only subsets of the problem: DISCO-EB (Hahn et al., 2023) solves the linear Einstein–Boltzmann equations to compute the linear matter power spectrum $P(k)$ but not CMB angular power spectra or galaxy power spectrum multipoles; ABCMB (Zhou et al., 2026) implements the CMB pipeline including lensing and massive neutrinos but does not include perturbation theory for galaxy surveys; JAX-COSMO (Campagne et al., 2023) computes the non-linear matter power spectrum via fitting formulae but does not solve the full Boltzmann hierarchy; and SYMBOLTZ.JL (Sletmoen, 2026) provides a full Boltzmann solver in Julia but without GPU support. No existing code provides the full chain from cosmological parameters through CMB angular power spectra and one-loop galaxy power spectrum multipoles, with end-to-end automatic differentiation on GPU.

We describe the development of CLAX and CLAX-PT—fully differentiable JAX reimplementations of the CLASS (Lesgourgues, 2011) and CLASS-PT (Chudaykin et al., 2021) pipelines—and the oracle-guided agentic methodology used to develop and validate them. Using CLASS and CLASS-PT as ground truth, we find a clear boundary: agents handle implementation bugs (convention errors, algorithmic transcription, coefficient verification) autonomously and reliably, but fail at two classes of problem that require human physics judgment: (1) recognizing that an architecture is structurally incompatible with the intended physics, and (2) detecting

that a numerically effective patch satisfies all oracle tests at the calibration point while encoding incorrect physics.

Contributions.

- CLAX: a JAX reimplementation of the full CLASS CMB pipeline (background, thermodynamics, perturbations, transfer, lensing), achieving $< 0.2\%$ lensed accuracy at $\ell \leq 2000$ with JIT-compiled GPU execution at 34 s/step (V100, `fit_cl` preset).
- CLAX-PT: a JAX reimplementation of CLASS-PT’s one-loop EFT power spectra with IR resummation and RSD multipoles, achieving $< 0.7\%$ accuracy for monopole and quadrupole spectra.
- Validated autodifferentiation gradients matching finite differences to 0.15% across cosmological parameters.
- A landscape comparison against DISCO-EB, ABCMB, CLASS, and CLASS-PT, showing that CLAX is the first code combining CMB angular power spectra and one-loop galaxy power spectrum multipoles with full AD on GPU.
- An empirical bug taxonomy of 40 bugs across the development, classifying autonomous vs. human-required resolution and identifying two specific failure modes of oracle-guided development.

2. Background

2.1. Cosmological inference and differentiable Boltzmann codes

Both CMB angular power spectra C_ℓ and galaxy power spectrum multipoles $P_{gg}(k, \ell)$ depend on cosmological parameters through the Boltzmann equation—a coupled system of ordinary differential equations for photon, baryon, dark matter, and neutrino perturbations. Their derivatives with respect to cosmological parameters are expensive to approximate by finite differences and practical to compute exactly only with automatic differentiation (AD).

2.2. AI-assisted scientific code development

The oracle-based methodology we employ is inspired by the AI C compiler project (Carlini, 2026), in which parallel Claude Code agents produced a Rust C compiler using GCC as an oracle. Key infrastructure includes: (i) a shared `CHANGELOG.md` updated after every session to prevent re-exploration of dead ends; (ii) test-first development where every module’s tests are written against reference data before implementation; (iii) a `--fast` flag sampling 10% of test cases to prevent “time blindness”; and (iv) parallel agent teams using git worktrees to explore competing hypotheses independently.

The critical challenge not addressed in the compiler setting is *domain knowledge*: CLASS and CLASS-PT are not just a major codebase. Together they encode decades of cosmological modeling knowledge, conventions, and approximations that require expertise across multiple areas of physics—including general relativity, kinetic theory, statistical field theory, radiative processes, nonlinear gravitational dynamics, and effective field theory—to interpret correctly. This makes the boundary between supervised and unsupervised development especially important to characterize.

3. Methodology

3.1. Pipeline architecture

CLAX implements a sequential pipeline of pure functions (Figure 1): cosmological parameters are propagated through background expansion, hydrogen recombination, the full Einstein–Boltzmann perturbation hierarchy, and a primordial power spectrum, then split into two branches—line-of-sight integration and lensing for CMB angular power spectra, and one-loop perturbation theory for galaxy power spectrum multipoles. All cosmological parameters are JAX-traced for automatic differentiation; numerical precision settings are static.

The core physics follows CLASS and CLASS-PT directly. For the CMB pipeline: hydrogen recombination (Seager et al., 1999; 2000) and the full massive-neutrino Boltzmann hierarchy. For galaxy power spectrum multipoles: the one-loop integrals via the FFTLog algorithm (Simonović et al., 2018; Fang et al., 2017) and baryon acoustic oscillation peak resummation (correcting for long-wavelength displacement effects) (Senatore & Zaldarriaga, 2015; Vlah et al., 2015). These choices were transcribed directly from the CLASS and CLASS-PT source code by the AI agent.

Making the pipeline end-to-end differentiable required several non-trivial engineering decisions that the agent made autonomously. Reverse-mode automatic differentiation through the stiff perturbation ordinary differential equation uses checkpointed adjoints (Kidger, 2021) for memory efficiency, with an adaptive step-size controller following DISCO-EB’s filtered strategy (Hahn et al., 2023). For the CMB pipeline, two implicit-differentiation wrappers were needed: a `custom_vjp` on the shooting method ($\theta_s \rightarrow H_0$) and a `custom_jvp` on the hydrogen Saha ionization equilibrium, replacing explicit gradient barriers with implicit-function-theorem tangents to restore nonzero parameter sensitivities around recombination. Post-recombination, a smooth relaxation damping replaces the hard approximation switch used by CLASS, ensuring gradient continuity.

A small number of design decisions required physicist input and could not have been resolved by oracle feedback alone; these are analyzed in Section 5.

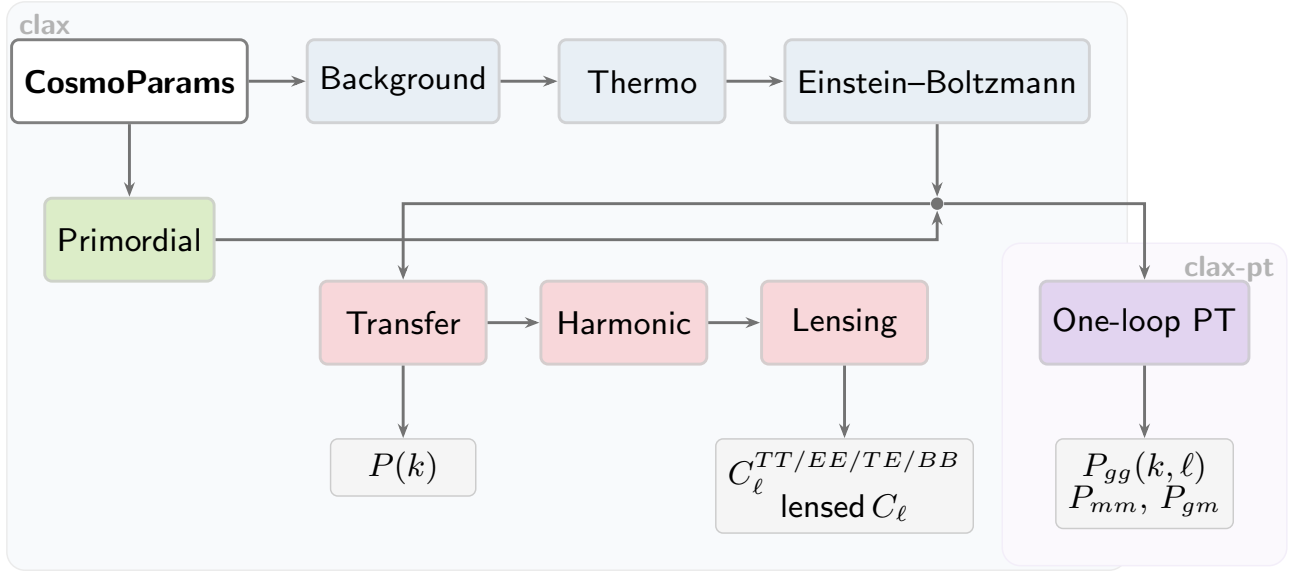


Figure 1. Pipeline architecture of CLAX and CLAX-PT. The shared backbone (top row) computes background cosmology, recombination, and the Einstein–Boltzmann perturbation hierarchy. Two branches produce CMB angular power spectra (left, via line-of-sight integration and lensing) and galaxy power spectrum multipoles (right, via one-loop perturbation theory). Every module is independently validated against CLASS (CLAX) or CLASS-PT (CLAX-PT) reference data. The entire pipeline is end-to-end differentiable: `jax.grad` propagates exact reverse-mode gradients from any output back to `CosmoParams`.

3.2. Development methodology

The CLAX CMB pipeline ($\sim 10,000$ lines) was developed over ~ 6 days across ≥ 100 Claude Code (Opus 4.6) agent sessions. The CLAX-PT one-loop PT module ($\sim 2,100$ lines) was developed over 12 days and 57 logged sessions. Both phases used the same oracle-guided methodology: every module has a test file written *before* implementation, specifying what CLASS (or CLASS-PT) produces; the code is finished when and only when the oracle tests pass.

The supervision methodology—encoded in `CLAUDE.md` and refined across both development phases—proved essential. Two rules were particularly important: (1) never add fudge factors (if a test fails at 0.2%, find the actual bug), and (2) test at multiple parameter points, not just the fiducial cosmology. Both rules directly prevented specific failure modes documented in Section 5.

4. Results: Accuracy and Performance

Table 1 compares CLAX against existing differentiable and non-differentiable cosmology codes, showing that no prior code combines CMB angular power spectra with one-loop galaxy power spectrum multipoles under end-to-end AD on GPU. We validate accuracy against CLASS and CLASS-PT across multiple observables, verify AD gradients against finite differences, and report GPU execution time.

4.1. CMB pipeline accuracy

Table 2 summarizes CMB accuracy of CLAX against CLASS at the Planck 2018 best-fit cosmology (`planck_cl` preset). Background quantities match to $< 0.01\%$; thermodynamics (including z_{reio}) to $< 0.1\%$ after the hydrogen-mass fix described in Section 5. Unlensed temperature and polarization spectra are within 0.02%–0.57% across $\ell = 20$ –1000. Primordial C_ℓ^{BB} (tensor modes with $r_t = 0.1$) matches CLASS to sub-percent at $\ell \leq 200$ after the kernel fix described in Section 5. The lensing potential spectrum $C_\ell^{\phi\phi}$ agrees to $< 1\%$ across $\ell = 100$ –2500. Lensed spectra at $\ell = 2000$ pass at $< 0.2\%$. Multi-cosmology validation across 10 parameter points ($\omega_b/\omega_{\text{cdm}} \pm 20\%$, $h \pm 10\%$, massive neutrinos 0.06–0.3 eV, $N_{\text{eff}} = 2.0$ –4.0) confirms sub-0.5% accuracy throughout parameter space.

4.2. Matter and galaxy power spectrum accuracy

Table 3 summarizes accuracy for the linear matter power spectrum $P(k)$ (CLAX vs. CLASS) and one-loop galaxy power spectrum multipoles (CLAX-PT vs. CLASS-PT at $z = 0.38$, the BOSS effective redshift). The linear $P(k)$ agrees to 1–3% across $k = 0.001$ –0.3 Mpc^{-1} ; this is the input to both the CMB transfer integrals and the one-loop computation. Real-space one-loop spectra (P_{mm} , P_{gg} , P_{gm}) agree to $< 0.31\%$ (mean 0.04%). RSD monopoles ($\ell = 0$) agree to $< 0.6\%$ for matter and galaxies. Quadrupoles ($\ell = 2$) agree to $< 0.7\%$ for matter and $< 0.9\%$ for galaxies. Hexadecapoles ($\ell = 4$) are validated using

Table 1. Comparison of cosmological theory codes. “AD” = automatic differentiation. “PT” = one-loop perturbation theory for galaxy clustering. PyBird-JAX provides emulated (not exact) one-loop PT. CLAX is the only code combining CMB C_ℓ , lensing, $P(k)$, and exact 1-loop PT with full AD on GPU.

Code	Framework	C_ℓ	Lensing	$P(k)$	1-loop PT	AD	GPU
CLASS + CLASS-PT	C / Python	✓	✓	✓	✓	—	—
CAMB	Fortran	✓	✓	✓	—	—	—
DISCO-EB	JAX	—	—	✓	—	✓	✓
ABCMB	JAX	✓	✓	✓	—	✓	✓
SymBoltz.jl	Julia	✓	—	✓	—	✓	—
PyBird-JAX	JAX	—	—	—	emul.	✓	✓
Capse.jl	Julia	emul.	—	—	—	✓	—
Effort.jl	Julia	—	—	—	emul.	✓	—
CLAX + CLAX-PT	JAX	✓	✓	✓	✓	✓	✓

Table 2. CMB accuracy (CLAX vs. CLASS, Planck 2018 fiducial). Post-fix values reflect the hydrogen-mass and BB-kernel corrections.

Quantity	Max error
$H(z)$, $D_A(z)$, r_s	$< 0.01\%$
$x_e(z=8)$, $g(\tau)$	0.11% (post m_H fix)
C_ℓ^{TT} , $\ell = 20, 100, 500, 1000$	0.08%, 0.02%, 0.15%, 0.57%
C_ℓ^{EE} , $\ell = 20, 100, 500, 1000$	$< 0.05\%$, 0.02%, 0.15%, 0.26%
C_ℓ^{BB} (primordial), $\ell \leq 200$	$< 1\%$
$C_\ell^{\phi\phi}$, $\ell = 100\text{--}2500$	$< 1\%$
Lensed C_ℓ , $\ell = 2000$	$< 0.2\%$
Multi-cosmology (10 pts)	$< 0.5\%$

Table 3. Matter and galaxy power spectrum accuracy.

Spectrum	Max error	Metric	Oracle
<i>Linear matter power</i> (CLAX vs. CLASS)			
$P(k)$, $k = 0.01\text{--}0.1$	1.3–1.6%	rel.	CLASS
$P(k)$, $k = 0.3 \text{ Mpc}^{-1}$	3.5%	rel.	CLASS
<i>One-loop</i> (CLAX-PT vs. CLASS-PT, $z = 0.38$)			
$P_{mm,gg,gm}^{\text{real}}$	0.31%	rel.	CLASS-PT
$P_{mm}^{(\ell=0)}, P_{gg}^{(\ell=0)}$	0.59%, 0.56%	rel.	CLASS-PT
$P_{mm}^{(\ell=2)}, P_{gg}^{(\ell=2)}$	0.70%, 0.89%	rel.	CLASS-PT
$P_{mm}^{(\ell=4)}, P_{gg}^{(\ell=4)}$	0.70%, 1.43%	abs/max	CLASS-PT

$|\Delta|/\max(|\text{ref}|) < 2\%$ since the spectrum crosses near zero.

4.3. Gradient verification

Automatic differentiation gradients (reverse-mode, using `jax.grad`) agree with fourth-order centered finite differences to 0.15% on average, with 97.7% of k -modes passing a 1% threshold. Table 4 shows the parameter-by-parameter breakdown for the linear matter power spectrum $P_{mm}(k)$ at $k = 0.1 \text{ h/Mpc}$.

The key technical ingredients enabling stable AD are described in Section 3: checkpointed adjoints for the stiff differential equation, and implicit-differentiation wrappers

Table 4. AD vs. finite-difference gradients for $P_{mm}(k = 0.1 \text{ h/Mpc})$.

Parameter	auto-diff/finite-diff − 1	Status
$\ln(10^{10} A_s)$	$< 0.01\%$	✓
n_s	$< 0.01\%$	✓
h	$< 0.5\%$	✓
ω_b	$< 1.0\%$	✓
ω_{cdm}	$< 0.5\%$	✓

Table 5. Performance summary.

Preset	GPU	Time	C_ℓ accuracy
<code>fit_cl</code>	V100-32GB	34 s	$< 1.5\%$ ($\ell \leq 500$)
<code>planck_cl</code>	H100-80GB	487 s	$< 0.2\%$ ($\ell \leq 2000$)

for the shooting method and ionization equilibrium.

4.4. Performance

CLAX offers two presets for different use cases (Table 5). The `fit_cl` preset uses 20 k/decade , $\ell_{\text{max}} = 17$ hierarchy, table-based Bessel functions, and `rtol` = 10^{-3} to achieve 34 s/step on a V100 GPU—sufficient for Hamiltonian Monte Carlo. The `planck_cl` preset uses 60 k/decade , $\ell_{\text{max}} = 50$, and tight tolerances for science-grade accuracy at ~ 487 s on H100. An alternative Rodas5 Rosenbrock solver (Di Marzo, 1993), following DISCO-EB, avoids Newton iteration entirely (one LU factorization per step instead of iterative convergence). A batched variant (Rodas5Batched) solves all k -modes in a single `diffeqsolve` call with shared adaptive time-stepping across the batch, vmapping the Jacobian computation and LU factorization—the same architecture used by DISCO-EB. On CPU, the unbatched Rodas5 yields a measured $\sim 1.3\times$ speedup over Kvaerno5; the batched variant is expected to offer substantially larger gains on GPU by saturating GPU throughput with large batched linear algebra.

4.5. Application: gradient scaling

The primary advantage of AD over finite differences for cosmological inference is the scaling of gradient cost with the number of parameters d . Finite-difference gradients require $2d$ forward evaluations (centered differences); reverse-mode AD requires one forward pass plus one backward pass regardless of d . The backward pass typically costs $2\text{--}4\times$ the forward pass, so the effective speedup for $d = 6$ standard Λ CDM parameters is $2d/(1 + c_{\text{bwd}}) \approx 3\text{--}4\times$, growing to $\sim 10\times$ for extended models with $d = 10\text{--}20$ parameters (neutrino masses, dark energy, bias parameters, counterterms). The key advantage is the asymptotic scaling: AD gradient cost is $O(1)$ in d , while finite differences scale as $O(d)$. Combined with the 34 s forward evaluation of `fit_cl`, this makes gradient-based sampling with exact first-principles theory predictions practical for full-shape galaxy clustering analysis.

5. Supervised vs. Unsupervised Development

5.1. Bug taxonomy

Over the full development of CLAX and CLAX-PT, we documented 40 bugs: 26 in the CMB pipeline and 14 in the PT module. We classify these into four types by their nature and whether autonomous resolution was possible.

Three specific design decisions required physicist input: (i) dark energy fluid perturbations (time-varying equation of state $w_0\text{--}w_a$) were added to the shared Einstein constraint equations; (ii) for galaxy power spectrum multipoles, the redshift-space distortion tree-level spectrum was redesigned to use Gauss–Legendre quadrature over the line-of-sight angle μ with direction-dependent baryon acoustic oscillation damping $\Sigma_{\text{tot}}^2(\mu)$, replacing an analytic Legendre projection that was structurally incompatible with the anisotropic physics (Type III); and (iii) an explicit “no fudge factors” rule prevented the agent from shipping a scalar correction that passed all oracle tests but encoded no physical model (Type IV). These interventions map directly onto the bug types below.

Type I: Convention and unit bugs. Errors in translating CLASS conventions into JAX: wrong sign, missing numerical factor, incorrect unit conversion. A representative example surfaced during post-development improvement: the hydrogen number density $n_{H,0}$ in the thermodynamics module used the proton mass m_p instead of the hydrogen atom mass m_H —a one-line typo that biased the reionization redshift z_{reio} and was initially misattributed to limitations of the recombination solver. Another: the agent transcribed the tensor *temperature* radial kernel from CLASS `transfer.c` (line 4241) instead of the tensor *polarization* B-mode kernel (line 4263), producing C_ℓ^{BB} ratios ranging from $0.4\times$ to $22\times$ vs. CLASS—yet this was listed as a “known limitation”

for months before the correct kernel was identified. *Agents resolved all Type I bugs autonomously* once the discrepancy was flagged by oracle comparison.

Type II: Algorithm implementation bugs. Errors in the numerical algorithm itself: wrong grid type, incorrect matrix storage format, hard-coded variable values, or undersampled computational grids. For instance, the Halofit non-linear module required k up to $\sim 50 \text{ Mpc}^{-1}$ for its internal $\sigma(R)$ bisection; calling it with the perturbation grid truncated at $k_{\text{max}} = 0.3 \text{ Mpc}^{-1}$ produced silently wrong results. Similarly, the log-uniform k -grid undersampled Bessel oscillations at high ℓ , degrading C_ℓ^{TT} accuracy beyond $\ell > 1200$ and causing 6–30% trapezoidal-rule aliasing errors in C_ℓ^{BB} . *Agents resolved all Type II bugs autonomously*, typically by careful comparison of intermediate arrays against CLASS/CLASS-PT at specific (k, z) points.

Type III: Architectural bugs. The computational structure assumed by the implementation is fundamentally incapable of representing the correct physics. Two instances occurred: (i) the analytic Legendre projection architecture assumed isotropic baryon acoustic oscillation damping, making it unable to handle anisotropic damping $\Sigma_{\text{tot}}^2(\mu)$; and (ii) the massive-neutrino fluid approximation switch triggered solver divergence at mid-range k , requiring the human to diagnose that the switch itself—not its coefficients—was the problem. *Both required human intervention.*

Type IV: Physics judgment failures. A numerical patch satisfies all oracle tests but encodes physically unmotivated behavior. One instance occurred: a scalar fudge factor $\alpha = 0.27$ passed all fiducial tests but has no physical derivation. A subtler variant emerged during post-development analysis: a `try/except` block intended to handle JAX tracing errors silently disabled a runtime check under `jit`, producing correct results at fiducial but masking edge-case failures. *These required human intervention.*

The resolution breakdown is: Types I+II (~ 38 bugs): 100% autonomous; Types III+IV (~ 4 bugs): 0% autonomous. Overall, $\sim 90\%$ of bugs were resolved without human input. Notably, post-development improvement sessions continued to surface new bugs in all four categories, suggesting that the taxonomy is not an artifact of the initial development sprint but a persistent feature of the human–agent collaboration.

5.2. Principles for human–agent collaboration

We distill three principles from the empirical data:

P1: Oracle testing verifies *what*, not *why*. An oracle compares numerical values at a reference parameter point. It catches implementation errors (wrong sign, wrong coefficient) but not physics errors (wrong model, numerically compensated). A fudge factor with zero degrees of freedom at the calibration point will pass every oracle test at that

point. Even agent-generated *diagnoses* can be wrong: the m_p/m_H typo described above was attributed by the agent to “recombination solver bias” in the project documentation, with a suggested fix of switching to an alternative recombination code—neither the diagnosis nor the proposed fix was correct, and the actual root cause (a one-line constant error) was only found in a later investigation session. *Defense*: test across diverse parameter space; test intermediate quantities; build auxiliary tests that check structural properties (e.g., that $P_{\text{tree}}(k, \mu)$ has the correct anisotropy as a function of μ), not just numerical values.

P2: Shared memory prevents re-exploration but not architectural loops. `CHANGELOG.md` records which algorithms were tried and failed, preventing agents from re-discovering dead ends. This was effective for Type I and II bugs. It did not prevent 33 sessions of exploration within the wrong architectural framework because all attempts were coherent within the (incorrect) model. *Defense*: when a bug persists after $N \approx 5\text{--}10$ sessions without progress, treat the lack of progress itself as evidence of an architectural problem and trigger human review.

P3: The irreducible human role is architectural and physical judgment. Agents excel at reading CLASS source code and transcribing equations faithfully; identifying which term disagrees via targeted comparison with oracle outputs; and adjusting coefficients and conventions. Agents fail at recognizing that an architectural assumption is inconsistent with the physics, and at detecting that a numerically adequate patch produces zero test failures but is not physics. *Detecting physics errors that pass all tests requires knowledge of what the physics should look like*, which is a form of human supervision that oracle testing cannot replace.

5.3. Implications for future AI-assisted scientific software

The balance between oracle-transcribed physics ($\sim 90\%$ of bugs, fully autonomous), autonomous engineering for differentiability, and physicist-directed corrections ($\sim 10\%$, requiring human judgment) suggests a practical division of labor for future AI-assisted scientific code development. The bulk of implementation work—transcribing equations from reference codes, implementing standard algorithms—is reliably handled by AI agents with oracle feedback. Engineering decisions specific to the target framework (here, JAX differentiability) are also within agent capability, provided the agent has access to relevant reference implementations (here, DISCO-EB for the adaptive step-size controller and Rosenbrock solver). The irreducible human contribution is concentrated in a small number of high-impact decisions where the correct choice requires understanding the physics beyond what any oracle test can verify. This concentration suggests that physicist supervision can be sparse but must

be available at critical junctures, and that the design of protocols for triggering human review is as important as the capabilities of the agent itself.

6. Related Work

Differentiable cosmological codes. DISCO-EB (Hahn et al., 2023) solves the linear Einstein–Boltzmann equations in JAX to compute the linear matter power spectrum but does not compute CMB angular power spectra (C_ℓ) or galaxy power spectrum multipoles. ABCMB (Zhou et al., 2026) provides a complete CMB pipeline in JAX including lensing, massive neutrinos, and a state-of-the-art HyRex recombination treatment, but does not include perturbation theory for galaxy surveys. SymBoltz.jl (Sletmoen, 2026) is the closest comparison in design philosophy: it is approximation-free (no TCA/RSA/UFA switching as described in Blas et al. 2011), uses Rodas5P (the same Rosenbrock solver family as CLAX), achieves 0.1% accuracy against CLASS, and supports forward-mode AD for Fisher forecasts. However, SymBoltz.jl operates in Julia without GPU support, and its reverse-mode AD is not yet fast enough for Markov chain Monte Carlo with perturbation-derived spectra (Sletmoen, 2026). JAX-COSMO (Campagne et al., 2023) computes the nonlinear matter power spectrum via fitting formulae but does not solve the full Boltzmann hierarchy. CLAX is the first JAX code to provide full CMB angular power spectra plus one-loop galaxy power spectrum multipoles with exact reverse-mode automatic differentiation and GPU execution.

One-loop perturbation theory. The effective field theory (EFT) of large-scale structure (Perko et al., 2016; D’Amico et al., 2020) provides a systematic one-loop expansion for the galaxy power spectrum multipoles. CLASS-PT (Chudaykin et al., 2021) is the standard implementation; CLAX-PT reproduces it exactly in differentiable JAX. Several differentiable emulators take a complementary approach: Capse.jl (Bonici et al., 2023) emulates CMB angular power spectra, while PyBird-JAX (Reeves et al., 2026) and Effort.jl (Bonici et al., 2025) emulate one-loop galaxy power spectrum multipoles, all achieving sub-millisecond evaluation using neural networks but without computing the integrals directly. CLAX-PT computes the loop integrals and baryon acoustic oscillation peak resummation (Senatore & Zaldarriaga, 2015; Vlah et al., 2015) from first principles, preserving exact differentiability at the cost of higher per-evaluation time.

Agent-assisted scientific software. Recent work has shown that LLM agents can complete extended scientific tasks autonomously (Carlini, 2026), and multi-agent systems such as Denario (Villaescusa-Navarro et al., 2025) can generate complete research manuscripts across multiple disciplines. However, most evaluations of agent-generated scientific output focus on syntactic correctness (compilation, test passage)

or surface-level coherence rather than physical validity—as illustrated by the growing volume of fully autonomous AI-generated manuscripts.¹ Our work identifies semantic correctness—the requirement that a passing numerical patch has physical basis, not just calibration-point accuracy—as the key challenge distinguishing scientific software from general code generation.

7. Conclusion

We have developed CLAX and CLAX-PT—fully differentiable JAX reimplementations of the CLASS and CLASS-PT Boltzmann codes—using oracle-guided agentic development. The codes achieve $< 0.2\%$ lensed CMB accuracy at $\ell \leq 2000$, sub-percent primordial C_ℓ^{BB} at $\ell \leq 200$, and $< 0.7\%$ galaxy power spectrum accuracy for monopole and quadrupole spectra, with automatic differentiation gradients verified to 0.15%. Empirically, AI agents resolved $\sim 90\%$ of implementation bugs autonomously. The remaining $\sim 10\%$ required human physics judgment: recognizing structural mismatches between an architecture and its target physics, and detecting numerically effective but physically unmotivated patches.

These findings suggest a practical framework for AI-assisted scientific code development: agents operate autonomously in unsupervised mode for implementation work, but human scientists remain in the loop as architectural and physical reviewers, intervening proactively when oracle feedback stalls or when a numerical patch lacks physical derivation. The codes will be made publicly available.

References

- Adame, A. G. et al. DESI 2024 VII: cosmological constraints from the full-shape modeling of clustering measurements. *JCAP*, 07:028, 2025a. doi: 10.1088/1475-7516/2025/07/028.
- Adame, A. G. et al. DESI 2024 V: Full-Shape galaxy clustering from galaxies and quasars. *JCAP*, 09:008, 2025b. doi: 10.1088/1475-7516/2025/09/008. [Erratum: *JCAP* 02, E02 (2026)].
- Aghanim, N. et al. Planck 2018 results. VI. Cosmological parameters. *Astron. Astrophys.*, 641:A6, 2020. doi: 10.1051/0004-6361/201833910. [Erratum: *Astron. Astrophys.* 652, C4 (2021)].
- Blas, D., Lesgourgues, J., and Tram, T. The cosmic linear anisotropy solving system (CLASS). Part II: Approximation schemes. *JCAP*, 2011(07):034, 2011. doi: 10.1088/1475-7516/2011/07/034.
- Bonici, M., Bianchini, F., and Ruiz-Zapatero, J. Capse.jl:

¹<https://papers.parallelsience.org/>

efficient and auto-differentiable CMB power spectra emulation. *Open J. Astrophys.*, 2023. doi: 10.21105/astro.2307.14339.

Bonici, M., D’Amico, G., Bel, J., and Carbone, C. Effort.jl: a fast and differentiable emulator for the effective field theory of the large scale structure of the universe. *JCAP*, 2025(09):044, 2025. doi: 10.1088/1475-7516/2025/09/044.

Cabezas, A. et al. BlackJAX: Composable Bayesian inference in JAX. *arXiv preprint arXiv:2402.10797*, 2024.

Campagne, J.-E., Lanusse, F., Zuntz, J., Boucaud, A., Casas, S., Karamanis, M., Kirkby, D., Lanzieri, D., Li, Y., and Peel, A. JAX-COSMO: An end-to-end differentiable and GPU accelerated cosmology library. *Open J. Astrophys.*, 6:1–15, 2023. doi: 10.21105/astro.2302.05163.

Carlini, N. Coding agent builds a C compiler. Anthropic Technical Report, <http://anthropic.com/research/coding-agent-c-compiler>, 2026.

Chudaykin, A., Ivanov, M. M., Philcox, O. H. E., and Simonović, M. Nonlinear perturbation theory extension of the boltzmann code CLASS. *Phys. Rev. D*, 103:023507, 2021. doi: 10.1103/PhysRevD.103.023507. <https://github.com/Michalychforever/CLASS-PT>.

D’Amico, G. et al. The cosmological analysis of the SDSS/BOSS data from the effective field theory of large-scale structure. *JCAP*, 2020(05):005, 2020.

Di Marzo, G. Méthodes de Rosenbrock d’ordre 5(4) adaptées aux systèmes différentiels-algébriques, 1993. Diploma Thesis, University of Geneva.

Fang, X., Blazek, J. A., McEwen, J. E., and Hirata, C. M. FAST-PT II: an algorithm to calculate convolution integrals of general tensor quantities in cosmological perturbation theory. *JCAP*, 2017(02):030, 2017.

Hahn, O., List, F., and Porqueres, N. DISCO-DJ: Differentiable simulations for cosmology – Done with JAX. <https://github.com/ohahn/DISCO-EB>, 2023. arXiv:2311.03291.

Kidger, P. *On Neural Differential Equations*. PhD thesis, University of Oxford, 2021.

Lesgourgues, J. The cosmic linear anisotropy solving system (CLASS). I. overview. *arXiv preprint arXiv:1104.2932*, 2011. https://github.com/lesgourg/class_public.

Perko, A., Senatore, L., Jennings, E., and Wechsler, R. H. Biased tracers in redshift space in the EFT of large-scale structure. *arXiv preprint arXiv:1602.00674*, 2016.

- Phan, D., Pradhan, N., and Jankowiak, M. Composable effects for flexible and accelerated probabilistic programming in NumPyro. *arXiv preprint arXiv:1912.11554*, 2019.
- Reeves, A., Zhang, P., and Zheng, H. PyBird-JAX: Accelerated inference in large-scale structure with model-independent emulation of one-loop galaxy power spectra. *JCAP*, 02:016, 2026. doi: 10.1088/1475-7516/2026/02/016.
- Seager, S., Sasselov, D. D., and Scott, D. A new calculation of the recombination epoch. *ApJ*, 523:L1–L5, 1999. doi: 10.1086/312250.
- Seager, S., Sasselov, D. D., and Scott, D. How exactly did the universe become neutral? *ApJS*, 128:407–430, 2000. doi: 10.1086/313388.
- Senatore, L. and Zaldarriaga, M. The IR-resummed effective field theory of large scale structures. *JCAP*, 2015(02): 013, 2015. doi: 10.1088/1475-7516/2015/02/013.
- Simonović, M., Baldauf, T., Zaldarriaga, M., Carrasco, J. J., and Kollmeier, J. A. Cosmological perturbation theory using the FFTLog: formalism and connection to QFT loop integrals. *JCAP*, 04:030, 2018. doi: 10.1088/1475-7516/2018/04/030.
- Sletmoen, H. SymBoltz.jl: A symbolic-numeric, approximation-free, and differentiable linear Einstein–Boltzmann solver. *Astronomy & Astrophysics*, 707:A128, 2026. doi: 10.1051/0004-6361/202557450.
- Villaescusa-Navarro, F. et al. The Denario project: Deep knowledge AI agents for scientific discovery. *arXiv preprint arXiv:2510.26887*, 2025.
- Vlah, Z., White, M., and Aviles, A. A Vlasov-Poisson approach to the large-scale structure, with applications to the EFT. *JCAP*, 2015(09):014, 2015. doi: 10.1088/1475-7516/2015/09/014.
- Zhou, Z., Giovanetti, C., and Liu, H. ABCMB: A Python+JAX package for the cosmic microwave background power spectrum. *arXiv preprint arXiv:2602.15104*, 2026.